



CAN(Controller Area Network) 테스트

[CRZ Technology](#)





Mango-M32F2 CAN 테스트

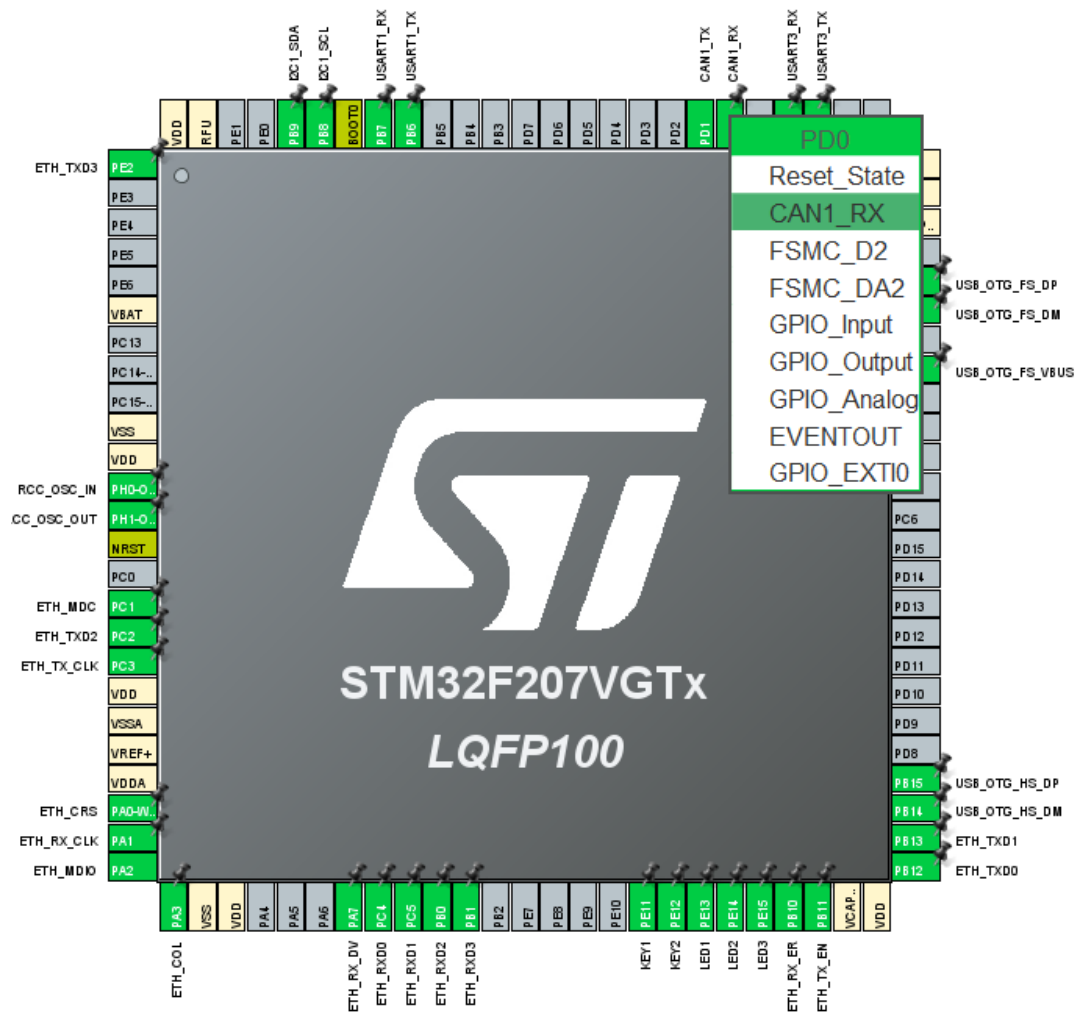
CAN pinmap

■ CAN – PD0, PD1

PD0	81	PD0	PD0	CAN1_RX
PD1	82	PD1	PD1	CAN1_TX
PD2	83	PD2	PD2	UART5_RX
	84	PD3		

Pinout view

■ PD0-CAN1_RX, PD1-CAN1_TX 선택.



CAN 설정

- “Connectivity” 클릭, “CAN1” 클릭.
- “Baud Rate” - “500000”, “Operating Mode” – “Normal Mode”.

The screenshot displays the STM32CubeMX software interface for configuring the CAN1 peripheral. On the left, the 'Connectivity' category is selected, and 'CAN1' is highlighted in the list of peripherals. The main window shows the 'CAN1 Mode and Configuration' settings. The 'Mode' section has the 'Activated' checkbox checked. The 'Configuration' section includes a 'Reset Configuration' button and tabs for 'Parameter Settings', 'User Constants', 'NVIC Settings', and 'GPIO Settings'. The 'Parameter Settings' tab is active, showing a table of bit timing parameters. The 'Basic Parameters' section shows various communication modes set to 'Disable'. The 'Advanced Parameters' section shows the 'Operating Mode' set to 'Normal'.

Bit Timings Parameters	
Prescaler (for Time Quantum)	4
Time Quantum	166.66666666666666 ns
Time Quanta in Bit Segment 1	6 Times
Time Quanta in Bit Segment 2	2 Times
Time for one Bit	1500 ns
Baud Rate	666666 bit/s
ReSynchronization Jump Width	1 Time

Basic Parameters	
Time Triggered Communication Mode	Disable
Automatic Bus-Off Management	Disable
Automatic Wake-Up Mode	Disable
Automatic Retransmission	Disable
Receive Fifo Locked Mode	Disable
Transmit Fifo Priority	Disable

Advanced Parameters	
Operating Mode	Normal

CAN 설정

- “GPIO” 클릭, “CAN” 클릭.
- “CAN_RX” - “Pull-up” 설정.

GPIO Mode and Configuration

Configuration

Group By Peripherals

I2C RCC USART USB NVIC
GPIO CAN ETH

Search Signals
Search (Ctrl+F) ☐ Show only Modified Pins

Pin	Signal on Pin	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PD0	CAN1_RX	n/a	Alternat...	Pull-up	High		☒
PD1	CAN1_TX	n/a	Alternat...	No pull-...	High		☐

PD0 Configuration :

GPIO mode: Alternate Function Push Pull

GPIO Pull-up/Pull-down: Pull-up

Maximum output speed: High

CAN 설정

- “Connectivity” 클릭, “CAN1” 클릭.
- “CAN1 RX0 interrupt” - “Enabled”

Search: [] [] []

Categories: A->Z

- System Core >
- Analog >
- Timers >
- Connectivity >
 - CAN1**
 - CAN2
 - ETH

CAN1 Mode and Configuration

Configuration

Reset Configuration

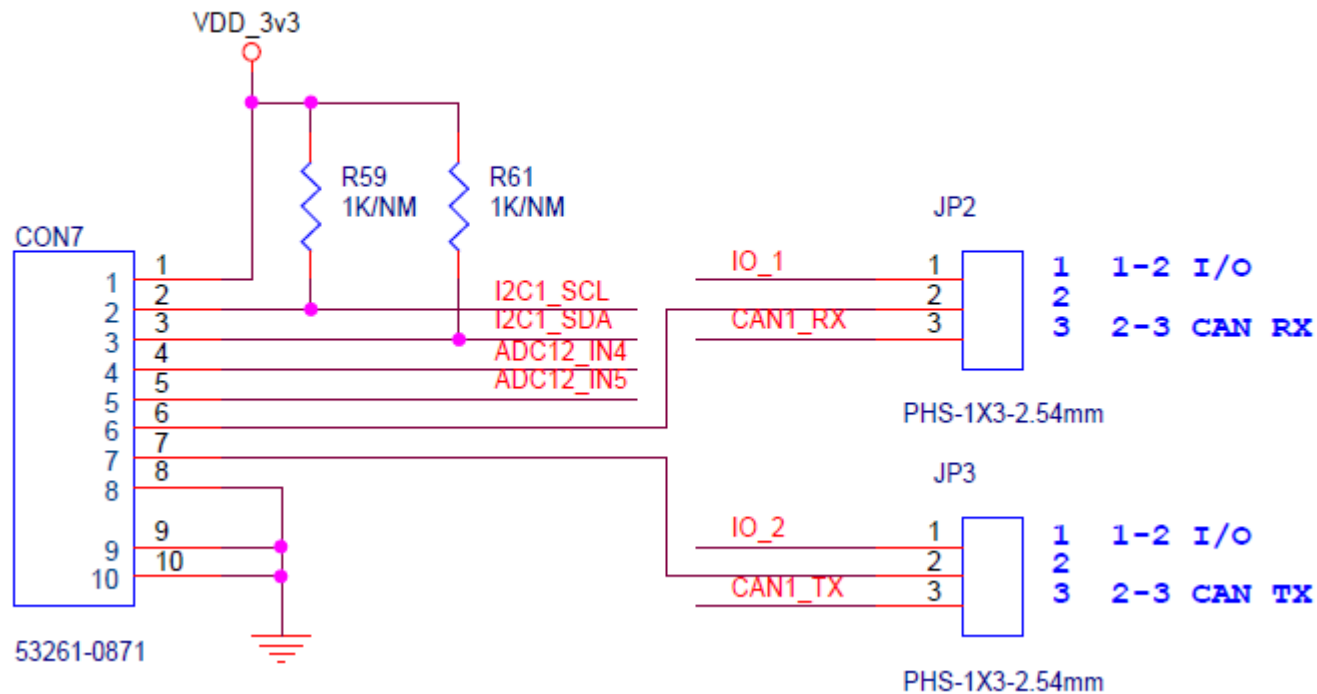
- User Constants
- NVIC Settings**
- GPIO Settings
- Parameter Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
CAN1 TX interrupts	☐	0	0
CAN1 RX0 interrupts	☒	0	0
CAN1 RX1 interrupt	☐	0	0
CAN1 SCE interrupt	☐	0	0

CAN 점퍼 설정

- CAN_RX – JP2 2-3 점퍼연결
- CAN_TX – JP3 2-3 점퍼연결

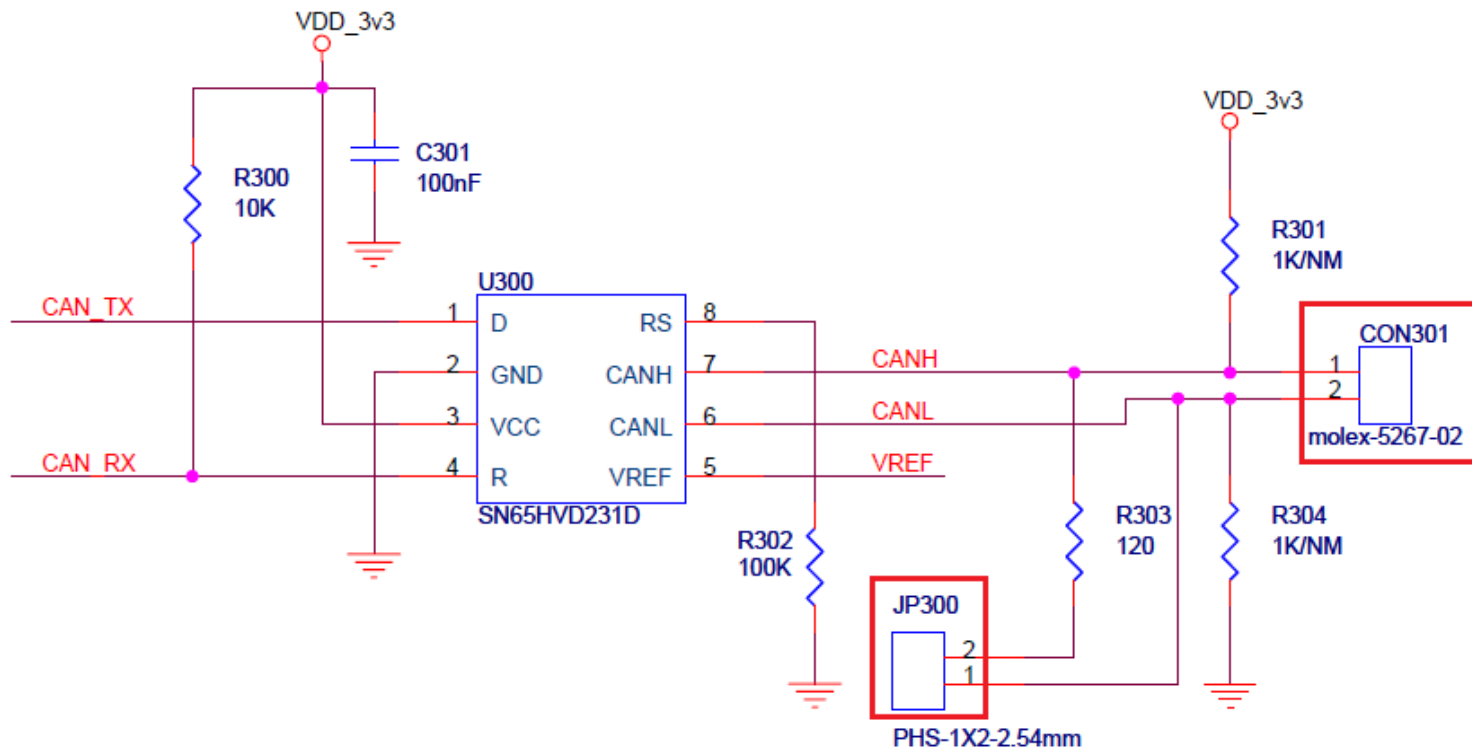
I2C CONNECTOR



CX_CAN – CAN Transceiver

- CAN 동작 시 CAN Transceiver 필요.
- 종단 저항 120옴 – JP300 점퍼 연결
- 2핀 케이블을 CON301에 접속하여 2개의 CX_CAN을 서로 연결.
- 8핀 케이블을 이용, Mango-M32F2(CON7)와 CX_CAN(CON31) 연결.

SN65HVD231D



CAN 테스트

- CAN TX – KEY1 / KEY2를 누를 때 CAN 메시지 전송.
- CAN RX – CAN 메시지 수신 시 LED1 / LED2 / LED3 토글.
- TX / RX 메시지를 시리얼 터미널에 디스플레이.

CAN TX

- 송신 메시지의 ID를 0x446으로 설정

```
/* USER CODE BEGIN 2 */  
  
HAL_CAN_Start(&hcan1);  
  
// Activate the notification  
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);  
  
TxHeader.DLC = 8; // data length  
TxHeader.IDE = CAN_ID_STD;  
TxHeader.RTR = CAN_RTR_DATA;  
TxHeader.StdId = 0x446; // ID  
/* USER CODE END 2 */
```

CAN TX

- KEY1 / KEY2가 눌리면 CAN 메시지를 송신.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == KEY1_Pin)
    {
        if(HAL_GPIO_ReadPin(KEY1_GPIO_Port, KEY1_Pin) == GPIO_PIN_RESET)
        {
            TxData[0] = 100;    // ms Delay
            TxData[1] = 40;     // loop rep
            TxData[2] = 0xaa;
            TxData[3] = 0xaa;
            TxData[4] = 0xaa;
            TxData[5] = 0xaa;
            TxData[6] = 0xaa;
            TxData[7] = 0xaa;

            HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox);
            printf("TX:%02x %02x %02x %02x %02x %02x %02x %02x\r\n", TxData[0],
        }
    }
}
```

CAN RX

- 수신 메시지 ID를 0x446으로 필터 설정.

```
/* USER CODE BEGIN CAN1_Init 2 */
CAN_FilterTypeDef canfilterconfig;

canfilterconfig.FilterActivation = CAN_FILTER_ENABLE;
canfilterconfig.FilterBank = 18; // which filter bank to use from the assigned ones
canfilterconfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;
canfilterconfig.FilterIdHigh = 0x446<<5;
canfilterconfig.FilterIdLow = 0;
canfilterconfig.FilterMaskIdHigh = 0x446<<5;
canfilterconfig.FilterMaskIdLow = 0x0000;
canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;
canfilterconfig.FilterScale = CAN_FILTERSCALE_32BIT;
canfilterconfig.SlaveStartFilterBank = 20; // how many filters to assign to the CAN1 (master can)

HAL_CAN_ConfigFilter(&hcan1, &canfilterconfig);
/* USER CODE END CAN1_Init 2 */
```

CAN RX

- RX callback을 활성화 하고 callback 함수에서 메시지를 수신.

```
// Activate the notification
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
```

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData);
    if (RxHeader.DLC == 8)
    {
        datacheck = 1;
    }
}
```

CAN RX

■ Main loop에서 수신 메시지 프린트.

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

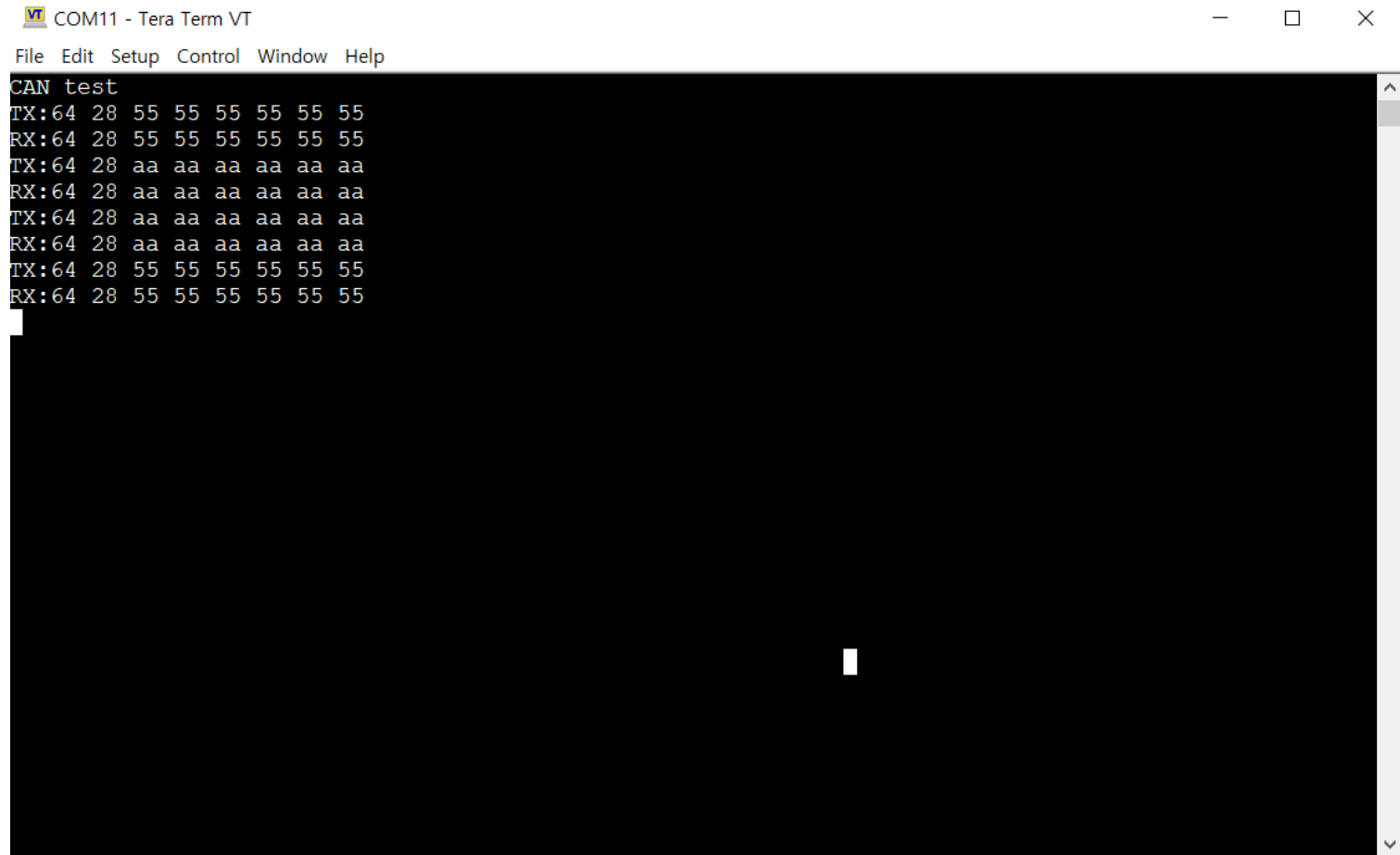
    /* USER CODE BEGIN 3 */
    if (datacheck)
    {
        printf("RX:%02x %02x %02x %02x %02x %02x %02x %02x\r\n", RxData[0],

            // blink the LED
            for (int i=0; i<RxData[1]; i++)
            {
                HAL_GPIO_TogglePin(GPIOE, LED1_Pin | LED2_Pin | LED3_Pin);
                HAL_Delay(RxData[0]);
            }

        datacheck = 0;
    }
}
/* USER CODE END 3 */
```

Test 결과 확인

■ CAN 송신 / 수신 메시지 프린트 확인.



The screenshot shows a Tera Term VT window titled "COM11 - Tera Term VT". The window has a menu bar with "File", "Edit", "Setup", "Control", "Window", and "Help". The main display area is black with white text showing the results of a CAN test. The text is as follows:

```
CAN test
TX:64 28 55 55 55 55 55 55
RX:64 28 55 55 55 55 55 55
TX:64 28 aa aa aa aa aa aa
RX:64 28 aa aa aa aa aa aa
TX:64 28 aa aa aa aa aa aa
RX:64 28 aa aa aa aa aa aa
TX:64 28 55 55 55 55 55 55
RX:64 28 55 55 55 55 55 55
```


Thank You

